UNIVERSIDAD DE EL SALVADOR
Facultad Multidisciplinaria De Occidente
Departamento de Ingeniería y Arquitectura
Ingeniería en Desarrollo de Software

Asignatura:

DESARROLLO Y REUTILIZACIÓN DE SOFTWARE

Ciclo X / Quinto Año

Tema:

Proyecto de Asignatura:
Tema N°5 : API RESTful y Singleton

Coordinadora de Cátedra:
Ing. Dinora Virginia de Melara

Tutora GT02:
Ing. Dinora Virginia de Melara

Alumnos:

Josué Edenilson Martínez Flores -- MF22036
Jose Arnoldo Landaverde Gomez -- LG22018
Rodrigo Vladimir Cruz Cortez -- CC22070
Nelson Josué Morán Mata -- MM19037
Axel Rodrigo Juarez Quevedo -- JQ22003

Fecha:
25/10/2026

Ingeniería en DESARROLLO DE SOFTWARE

Inicio -- Planteamiento de Idea

La empresa está organizada en cuatro áreas: Ventas, Producción, Transporte y Marketing, estas cuentan con un gerente que administra la información de sus empleados. Actualmente, cada gerente requiere:

- Consultar la lista de empleados. (Read/Get)
- Agregar nuevos empleados. (Create/Post)
- Actualizar datos existentes. (Update/Put)
- Eliminar registros. (Delete)

Mediante el desarrollo de una API REST con ASP.NET Core para poder interactuar con el archivo que contiene la información de los empleados, y a través de una clase Singleton encargada de gestionar en memoria la lista de empleados y su persistencia hacia el archivo JSON.

Desarrollo -- Implementación de API

Se estableció la base de la API RESTful para la gestión de empleados utilizando ASP.NET Core y .NET 10. El modelo "Empleado" fue definido con las propiedades "Id", "Nombre", "Cargo" y "Area", mientras que "EmpleadosController" se configuró para manejar las operaciones CRUD mediante las rutas correspondientes. La lógica para gestionar los empleados se implementó en la clase "EmpleadoSingletonService", que implementa la interfaz "IEmpleadoService".

La gestión de los empleados se implementó mediante el patrón Singleton, utilizando "EmpleadoSingletonService" para administrar los datos en memoria y mantener su persistencia en el archivo JSON. Además, se configuraron los controladores y OpenAPI/Swagger para facilitar la consulta y prueba de los diferentes endpoints de la API.

Como parte de la validación, se incorporaron las cuatro áreas establecidas para el contexto de la empresa: Ventas, Producción, Transporte y Marketing, junto con las validaciones necesarias para los datos recibidos.

También se ajustó la configuración de ejecución HTTP para facilitar las pruebas en el entorno local y se verificó la compatibilidad de las dependencias utilizadas.

Desarrollo -- Implementación de Servicios

Se estableció implementar servicios para centralizar la lógica de la gestión de los empleados, separándolo del controlador establecido, con la finalidad de que las operaciones CRUD no se implementan directamente en el controlador. En estos servicios se desarrollan dos archivos: IEmpleadoService y EmpleadoSingletonService, en donde IEmpleadoService es una interfaz que define el comportamiento esperado de los métodos de control de empleados, con métodos como obtener todos los empleados, crearlos, actualizarlos, borrarlos y validar áreas, es decir se definen los métodos CRUD para gestionar usuarios/empleados. Mientras que EmpleadoSingletonService es en la que contiene la lógica necesaria para poder llevar a cabo estas operaciones, además de registrarlo con patrón Singleton para establecer una única instancia.

Este servicio establece los métodos definidos en la interfaz, sin embargo, añade mecanismos de control de errores como lo es ILogger que registrar información y errores, SemaphoreSlim para controlar el acceso concurrente a la lectura y escritura del archivo JSON, además de métodos de lectura y escritura de un archivo JSON permitiendo conservar la información entre ejecuciones de la aplicación

Reutilización del código

El sistema se apoya en una API REST construida con ASP.NET Core que aplica el patrón de diseño Singleton para centralizar la gestión de los empleados y el acceso al archivo JSON compartido. Dado que el proyecto tiene una complejidad moderada, la reutilización de código resulta ser baja, aunque se refleja en la estructura del servicio, la interfaz y el controlador.

Interacción del sistema

El proyecto implementa una arquitectura en capas con ASP.NET Core, separando las responsabilidades de la API en controladores, servicios y modelos. Al ser una aplicación multicapa, la capa de presentación está representada por EmpleadosController ésta se encarga de recibir las solicitudes HTTP y devolver las respuestas correspondientes. La capa de servicios utiliza IEmpleadoService y EmpleadoSingletonService para centralizar las operaciones CRUD y aplicar el patrón Singleton, permitiendo compartir una única instancia del servicio dentro del proceso de la aplicación. Por otra parte, está el modelo Empleado define la estructura de los datos y el archivo empleados.json sirve como mecanismo de persistencia cuando se implementan las operaciones de lectura y escritura. Esta organización facilita el mantenimiento, la separación de responsabilidades y la reutilización del código.